Code Similarity Platform

Technical Specification & Source Code Documentation

Version: 1.0

Prepared by: Group 16 — Muhammad Nabeel Waheed, Thomas Neal, Ghassan Balouze, Kev Akpinar, Abdel Zahran, Ty Mabee

Kool Group Name: Last Minute Warriors

Course: COSC 4P02 — Course Project

2026-04-24

Contents

Chapter 1. System Overview

1.1 Purpose and Scope

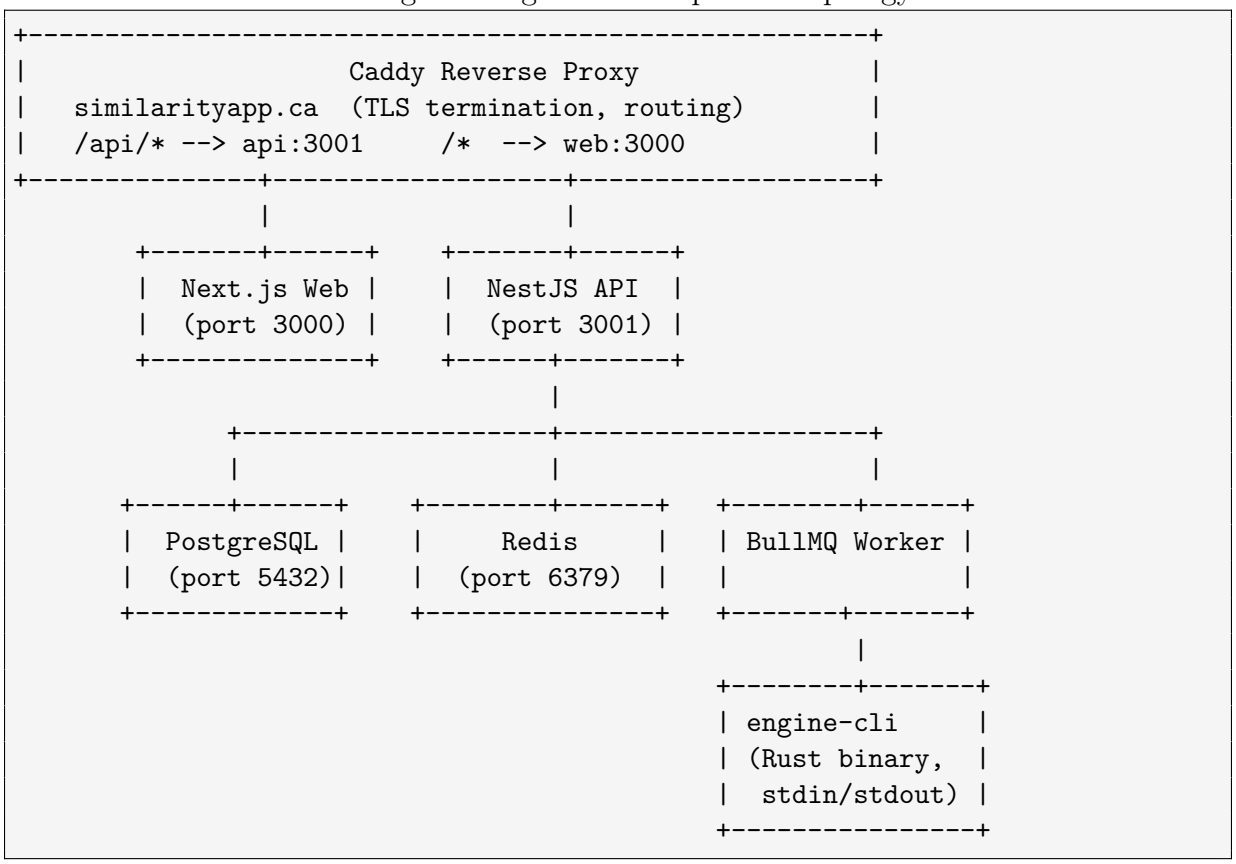
The Similarity App is a web-based source-code similarity and plagiarism review platform originally developed for Brock University COSC 4P02. It enables professors to create assignments, collect student source-code submissions, and run automated pairwise similarity analysis across all submissions. The results are presented in an interactive viewer that highlights matched code regions side-by-side.

The platform supports Java, C, and C++ source files. Similarity detection is powered by a custom Rust engine that implements a Greedy String Tiling (GST) algorithm operating over language-normalised token streams produced by tree-sitter parse trees.

1.2 High-Level Architecture

The system comprises five runtime components and a shared library package:

Listing 1.1: High-level component topology



1.3 Key Design Principles

- **Monorepo with npm workspaces**: A single repository hosts the web app, API, worker, shared library, engine, and all Prisma schema/migrations. This simplifies dependency management and ensures type contracts are shared across services.
- **Asynchronous job processing**: Upload preparation and comparison runs are decoupled from the HTTP request cycle via BullMQ queues backed by Redis. This avoids request timeouts for large batch operations.
- **Process-boundary engine isolation**: The Rust analysis engine runs as a separate process (spawned by the worker), communicating via JSON over stdin/stdout. This means the engine can crash without bringing down the Node.js worker, and enables independent versioning.
- **Token normalisation before comparison**: Identifiers, literals, and language boilerplate are normalised or discarded prior to similarity analysis. This prevents superficial renaming from evading detection.
- **Privacy-preserving anonymous submissions**: Students provide an RSA-OAEP-256-encrypted identity blob. The professor can decrypt it on demand; the server never stores cleartext identity unless the professor explicitly reveals it.
- **Filesystem object storage**: All uploaded archives and extracted source files are stored as flat files on a volume shared by the API and worker. An object-key abstraction provides path sanitisation and UUID-namespaced keys.

Chapter 2. Architecture Deep Dive

2.1 Frontend — `apps/web`

2.1.1 Responsibility

The web application provides the user interface for professors (assignment management, upload monitoring, comparison runs, pair viewer, settings) and for students (submission portal, status check, help page).

2.1.2 Technology and Structure

Built with Next.js 15 (App Router), React 19, TailwindCSS 4, TanStack React Query 5, and Monaco Editor. All API calls are made via a centralised client (`apps/web/lib/api.ts`) that prepends the `NEXT_PUBLIC_API_BASE_URL` base URL and sets `credentials: "include"` so session cookies are forwarded.

Key directories and files:

- `app/` — Next.js App Router pages: `/login`, `/signup`, `/student`, `/professor`, `/professor/assignments/[assignmentId]`, `/professor/assignments/[assignmentId]/pairs/[pairId]`, `/professor/settings`, `/professor/help`, `/help/student`.
- `components/` — Reusable React components including `professor-dashboard.tsx`, `assignment-workspace.tsx`, `evidence-viewer.tsx`, `pair-viewer-panel.tsx`, `student-submission-panel.tsx`, `submission-identity-panel.tsx`.
- `lib/api.ts` — Typed API client functions.
- `lib/submission-identity.ts` — Client-side RSA-OAEP-256 encryption of student identity before upload.
- `components/auth-hooks.ts` — React hooks for current-user session.

2.1.3 Interface to Other Components

The frontend communicates exclusively with the API over HTTPS (or HTTP in development) using REST. It does not connect directly to Redis, PostgreSQL, or the engine.

2.2 API Server — `apps/api`

2.2.1 Responsibility

The API is the HTTP gateway for all authenticated operations. It receives file uploads, enqueues background jobs, and serves query results from the database. It is the only component that talks to both the database and Redis (via BullMQ) synchronously from the HTTP path.

2.2.2 Technology and Structure

Built with NestJS 11 on top of Fastify 5 (instead of Express) for higher throughput and native multipart support. Prisma 6 is used as the ORM. The module graph is:

Listing 2.1: NestJS module graph

```
AppModule
  |- PrismaModule      (global singleton PrismaClient)
  |- HealthModule      (GET /health/live, GET /health/ready)
  |- QueueModule       (BullMQ queue publisher)
  |- AuthModule        (SessionAuthGuard, RolesGuard)
  |- AssignmentsModule (CRUD on assignments)
  |- SubmissionsModule (upload handling, identity reveal, downloads)
  |- ComparisonsModule (create and query comparison runs and pair results)
```

2.2.3 Authentication Guards

The `SessionAuthGuard` is applied globally. It reads the session cookie, hashes the token with SHA-256, looks it up in the `Session` table, and injects the user object onto the request. Routes marked `@Public()` bypass the guard. The `RolesGuard` enforces professor/student role constraints at the controller level.

2.2.4 Interface to Other Components

- **PostgreSQL**: via `PrismaService` for all persistent reads and writes.
- **Redis**: via BullMQ's `Queue` classes inside `QueueService` for job publishing only (no consuming).
- **Filesystem**: via the shared `storage.ts` module for staging and promoting uploaded archive files.

2.3 Background Worker — `apps/worker`

2.3.1 Responsibility

The worker processes two BullMQ queues:

1. `upload-preparation`: Extracts ZIP archives, parses source files, concatenates them, and persists `Submission` and `AssignmentTemplate` records.
2. `comparison-run`: Generates pair lists, invokes the engine binary, and persists `PairResult` and `PairMatch` records.

2.3.2 Technology and Structure

Plain Node.js with BullMQ 5 (consumes jobs), `fflate` (ZIP decompression), and ioredis (Redis connection). Has no HTTP server. Key services:

- `upload-preparation.processor.ts` — Job handler for the upload queue.

- `comparison-run.processor.ts` — Job handler for the comparison queue.
- `archive-extraction.service.ts` — Recursive ZIP extraction with depth/size/file-count guards.
- `concat.service.ts` — Deterministic source file concatenation with source map generation.
- `engine-runner.service.ts` — Spawns `engine-cli` binary, writes JSON to stdin, reads JSON from stdout.
- `pair-generation.service.ts` — Generates current×current and current×historical pairs.
- `persist-comparison-results.service.ts` — Upserts engine results into the database.
- `persist-prepared-upload.service.ts` — Writes prepared submission records.

2.3.3 Interface to Other Components

- **Redis**: consumes jobs via BullMQ Worker.
- **PostgreSQL**: reads submission/assignment data, writes results.
- **Filesystem**: reads staged archives, writes extracted source files.
- **Engine binary**: spawns as child process via `spawn()`, communicates via JSON over stdio.

2.4 Analysis Engine — `engine/`

2.4.1 Responsibility

The Rust binary `engine-cli` accepts an `AnalysisRequest` JSON payload on stdin and writes an `AnalysisResponse` JSON to stdout. It performs pairwise source-code similarity analysis using the Greedy String Tiling algorithm.

2.4.2 Crate Structure

Crate	Type	Responsibility
`engine-cli`	binary	stdin→stdout JSON wrapper
`engine-contracts`	library	shared serialisable types (request/response)
`engine-core`	library	analysis orchestration, scoring, comment matching
`engine-gst`	library	Greedy String Tiling algorithm
`engine-language`	library	tree-sitter parsing, token normalisation

2.4.3 Key Algorithms

Greedy String Tiling (GST): Implemented in `engine-gst/src/lib.rs`. Given two token sequences and a minimum match length, it iteratively finds the longest common unmarked substrings, marks them, and repeats until no qualifying match remains. This produces a set of *tiles* representing matched regions.

Token normalisation (engine-language): tree-sitter parse trees are walked. Identifiers become `ID`, type identifiers become `TYPE_ID`, string literals become `STR`/`STR_LIT`, numeric literals become `NUM`/`INT_LIT`/`FLOAT_LIT`, Java package/import declarations and C preprocessor directives are skipped entirely. This makes the comparison robust to renaming.

Template subtraction (engine-core): If a professor provides a template, the engine first runs GST between each submission and the template. Token regions that match the template with at least `TEMPLATE_MASK_THRESHOLD`=10 tokens are masked out before the student×student comparison proceeds.

Cluster merging (engine-core): Small GST tiles (below the per-assignment meaningful threshold of 10–15 tokens) that appear in close, symmetric proximity on both sides can be merged into a *cluster region*. This captures distributed copying such as restructured loops.

Similarity scoring (engine-core): A composite score is computed from harmonic/geometric token coverage, a contiguity factor (rewarding long contiguous runs), a fragmentation factor (penalising scattered matches), and a purity factor (ratio of exact-matched to effective tokens). The final score is in $[0, 1]$.

Comment comparison: Comments are normalised to lowercase alphanumeric words and matched exactly across both submissions. Template comments are subtracted. Comments shorter than `MEANINGFUL_COMMENT_MATCH_LENGTH`=35 characters are excluded.

2.4.4 Interface to Other Components

The engine is a standalone binary. It has no network interface, no database connection, and no shared memory. Communication is entirely via JSON on stdin/stdout.

2.5 Database Layer — `prisma/`

The canonical schema lives in `prisma/schema.prisma`. Migrations are in `prisma/migrations/`. Data access in both the API and worker is via the Prisma Client (generated at build time).

2.6 Shared Package — `packages/shared`

Provides TypeScript types and utility functions shared between the API and worker:

- `domain.ts` — `UserRole`, `AssignmentLanguage`, `UploadPurpose`, `SubmissionKind`, `MatchKind`, `SourceMapEntry`.
- `engine.ts` — `EngineRunRequest`, `EngineRunResponse` and associated types.
- `queues.ts` — Queue name constants (`upload-preparation`, `comparison-run`).

- `storage.ts` — Filesystem object-storage helpers: key generation (UUID-namespaced), path sanitisation, staged-object promotion.
- `uploads.ts` — Upload-related type helpers.

2.7 Data Flow: End-to-End Request Lifecycle

The following traces a complete professor workflow from assignment creation to viewing similarity results.

1. **Login**: Professor POSTs credentials to `POST/auth/login`. API verifies password hash (bcrypt), creates a `Session` row with SHA-256(token), sets an httpOnly cookie.
2. **Assignment creation**: Professor POSTs `POST/assignments` with title, language, and optional due date. API creates an `Assignment` row and an `AssignmentKey` row (a random public key students use to submit).
3. **Archive upload**: Professor uploads a ZIP via `POST/uploads/archive` (multipart). API stages the file, creates an `UploadBatch` record with status `RECEIVED`, and enqueues an `upload-preparation` job.
4. **Upload preparation (worker)**: Worker dequeues the job. It extracts the ZIP (fflate), traverses entries, identifies source files matching the assignment language, concatenates them deterministically with a source map, writes `Submission` and `SubmissionFile` records, and marks the `UploadBatch` as `READY`.
5. **Comparison run**: Professor POSTs `POST/comparison-runs`. API validates all batches are `READY`, creates a `ComparisonRun` record (`QUEUED`), and enqueues a `comparison-run` job with all submission sources and the active template (if any).
6. **Comparison execution (worker)**: Worker dequeues the job. It generates pairs (all current×current and current×historical), constructs an `EngineRunRequest`, spawns `engine-cli`, pipes JSON to stdin. Engine returns JSON with pair results. Worker marks the run `COMPLETED` and upserts `PairResult` and `PairMatch` rows.
7. **Results retrieval**: Professor fetches `GET/comparison-runs/:id` to get ranked pair results, then `GET/comparison-runs/pair/:pairId` to get the full match detail (byte spans, line positions, token ranges, concatenated source).
8. **Pair viewer**: Frontend renders left/right Monaco editor panes with highlighted match regions using byte-span data from the pair result response.

Chapter 3. Technology Stack

Layer	Technology	Version	Purpose
Frontend	Next.js	15.x	React framework with App Router
Frontend	React	19.x	UI component library
Frontend	TailwindCSS	4.x	Utility-first CSS
Frontend	TanStack React Query	5.x	Server state management and caching
Frontend	Monaco Editor	0.52.x	Code display with syntax highlighting
Frontend	Lucide React	1.7.x	Icon library
API	NestJS	11.x	Opinionated Node.js framework (DI, guards, modules)
API	Fastify	5.x	HTTP server (replaces Express for performance)
API	@fastify/cookie	11.x	Cookie parsing
API	@fastify/multipart	9.x	Multipart file upload
API & Worker	Prisma	6.x	Type-safe ORM with migration support
API & Worker	BullMQ	5.x	Redis-backed job queue
API & Worker	ioredis	5.x	Redis client
Worker	fflate	0.8.x	Pure-JS DEFLATE/ZIP decompression
Engine	Rust	1.86	Systems language for the analysis binary
Engine	tree-sitter	0.25.x	Incremental parser for Java, C, C++
Engine	serde / serde_json	1.x	JSON serialisation
Database	PostgreSQL	16	Relational database
Queue	Redis	7	Job queue and BullMQ state store
Reverse Proxy	Caddy	2	TLS termination, routing, gzip
Build	TypeScript	5.8.x	Static typing for all Node.js packages
Runtime	Node.js	22	JavaScript runtime

Note on Fastify vs Express: NestJS defaults to Express, but this project uses Fastify via `@nestjs/platform-fastify`. Fastify provides lower latency and native async plugin support, which is relevant for the file upload path where streaming is required.

Chapter 4. API & Interface Specifications

All API endpoints are served at the base path determined by `API_PORT` (default 3001). In production, the Caddy proxy strips the `/api` prefix before forwarding. Authentication is via an httpOnly session cookie named `similarity_session` (configurable).

4.1 Authentication Endpoints

Method	Path	Auth	Description
POST	`/auth/login`	Public	Email/password login. Sets session cookie.
POST	`/auth/professor-signup`	Public	Self-registration for professors. Sets session cookie.
POST	`/auth/logout`	Any authed	Revokes current session. Clears cookie.
GET	`/auth/me`	Any authed	Returns current user profile.
POST	`/auth/change-password`	Professor	Changes password after verifying current.
DELETE	`/auth/account`	Professor	Deletes own account and all owned data.

Login request body:

```
{ "email": "string", "password": "string" }
```

Login response (200):

```
{
  "userId": "string (cuid)",
  "email": "string",
  "role": "professor" | "student",
  "firstName": "string|null",
  "lastName": "string|null",
  "title": "string|null",
  "department": "string|null",
  "expiresAt": "ISO8601 datetime"
}
```

4.2 Assignment Endpoints

All require `professor` role.

Method	Path	Auth	Description
POST	`/assignments`	Professor	Create new assignment.

GET	`/assignments`	Professor	List all assignments for current professor.
GET	`/assignments/:assignmentId`	Professor	Get single assignment detail.
DELETE	`/assignments/:assignmentId`	Professor	Delete assignment and all owned data.
PATCH	`/assignments/:assignmentId/due-date`	Professor	Update due date.

Create assignment request body:

```
{
  "title": "string",
  "language": "JAVA" | "C" | "CPP",
  "dueDate": "ISO8601 datetime (optional)"
}
```

4.3 Submission / Upload Endpoints

Method	Path	Auth	Description
POST	`/uploads`	Professor	Create upload batch metadata record (pre-upload).
POST	`/uploads/archive`	Professor	Multipart: professor uploads ZIP archive for an assignment.
POST	`/uploads/student`	Student	Student submits source via API (authenticated).
POST	`/uploads/student/archive`	Student	Student uploads ZIP (authenticated).
GET	`/uploads/public/identity-key`	Public	Returns RSA public key for client-side encryption of identity.
POST	`/uploads/public/student/archive`	Public	Anonymous single-student ZIP upload with encrypted identity.
POST	`/uploads/public/student/bulk-archive`	Public	Anonymous bulk ZIP upload (professor-generated bulk).
GET	`/uploads/public/:uploadBatchId`	Public	Poll upload batch status by ID (with optional token).
GET	`/uploads/:uploadBatchId`	Authed	Get upload batch status (authenticated).
DELETE	`/uploads/assignment/:assignmentId/category/:category`	Professor	Delete all uploads of a given purpose category.
DELETE	`/uploads/assignment/:assignmentId/submissions/:submissionId`	Professor	Delete a specific submission.
DELETE	`/uploads/assignment/:assignmentId/templates/:templateId`	Professor	Delete a specific template version.
GET	`/uploads/assignment/:assignmentId/submissions/:submissionId/identity`	Professor	Reveal (decrypt) a submission's student identity.
GET	`/uploads/assignment/:assignmentId/submissions/:submissionId/download`	Professor	Download submission as ZIP.
GET	`/uploads/assignment/:assignmentId/templates/:templateId/download`	Professor	Download template as ZIP.
GET	`/uploads/assignment/:assignmentId/download`	Professor	Download all submissions as ZIP.
GET	`/uploads/assignment/:assignmentId/submissions/:submissionId`	Professor	Get submission detail.
GET	`/uploads/assignment/:assignmentId/templates/:templateId`	Professor	Get template detail.

Multipart archive upload fields:

```
-- Form fields (required):
assignmentId: string    (professor uploads)
purpose: "student_submission" | "historical_submission" | "template_upload"
assignmentKey: string   (student uploads -- references AssignmentKey.publicKey)
encryptedIdentity: string (public student upload -- sidenc:v1:... format)
-- File part:
archive: <binary zip>
```

4.4 Comparison Run Endpoints

All require `professor` role.

Method	Path	Auth	Description
POST	`/comparison-runs`	Professor	Create and enqueue a new comparison run.
GET	`/comparison-runs/:comparisonRunId`	Professor	Get comparison run status and ranked pair results.
GET	`/comparison-runs/pair/:pairResultId`	Professor	Get full pair result detail with match spans.

Create comparison run request:

```
{ "assignmentId": "string (cuid)" }
```

4.5 Health Endpoints

Method	Path	Auth and Description
GET	`/health/live`	Public. Liveness probe. Always returns 200 while process is running.
GET	`/health/ready`	Public. Readiness probe. Checks database and Redis connectivity.

Readiness response (200):

```
{ "status": "ok", "checks": { "database": "ok", "redis": "ok" } }
```

4.6 BullMQ Queue Interface

Two Redis-backed queues. The API publishes; the worker consumes.

Queue name	Job name	Payload

`upload-preparation`	`prepare-upload`	`UploadPreparationJob` (assignmentId, uploadBatchId, kind, language, optional pre-prepared data)
`comparison-run`	`run-comparison`	`ComparisonRunJob` (comparisonRunId, assignmentId, language, submissions array with concatenated source, optional template)

4.7 Engine Stdio Interface

The worker writes a JSON `AnalysisRequest` to the engine's stdin and reads a JSON `AnalysisResponse` from stdout.

AnalysisRequest schema (camelCase JSON):

```
{
  "schemaVersion": "1.0",
  "engineVersion": "string",
  "language": "java" | "c" | "cpp",
  "submissions": [
    {
      "submissionId": "string",
      "submissionKind": "current" | "historical",
      "source": "string (concatenated source)",
      "sourceMap": [{ "filePath": "string", "byteStart": N, "byteEnd": N }]
    }
  ],
  "template": { "source": "string", "sourceMap": [...] },  // optional
  "pairs": [
    { "pairId": "string", "leftSubmissionId": "string", "rightSubmissionId": "
    ↪ string" }
  ],
  "params": { "gstMinMatchLength": N, "minimumCommentLength": N }
}
```

AnalysisResponse schema:

```
{
  "schemaVersion": "1.0",
  "engineVersion": "string",
  "language": "java" | "c" | "cpp",
  "pairResults": [
    {
      "pairId": "string",
      "leftSubmissionId": "string",
      "rightSubmissionId": "string",
      "similarityScore": 0.0..1.0,
      "matchedTokenCount": N,
      "matches": [
        {
          "matchId": "string (code-N or comment-N)",
          "kind": "code" | "comment",
          "left": { "byteStart": N, "byteEnd": N, "filePath": "string|null",
                    "position": { lineStart, columnStart, lineEnd, columnEnd
    ↪ },
                    "tokens": { tokenStart, tokenEnd } },
          "right": { ... },
          "matchedTokenCount": N
        }
      ]
    }
  ]
}
```

Chapter 5. Database & Data Layer

5.1 Schema Overview

The database uses PostgreSQL 16. The schema is managed by Prisma 6. All primary keys are CUID strings.

5.2 Table Definitions

5.2.1 User

Column	Type	Constraints	Description
id	String (CUID)	PK	User identifier
email	String	UNIQUE, NOT NULL	Normalised to lowercase
firstName	String?		Optional display name
lastName	String?		
title	String?		Academic title (Prof., Dr., etc.)
department	String?		Academic department
passwordHash	String	NOT NULL	bcrypt hash
role	UserRole	NOT NULL	PROFESSOR or STUDENT
createdAt	DateTime	DEFAULT now()	
updatedAt	DateTime	@updatedAt	

5.2.2 Session

Column	Type	Constraints	Description
id	String	PK	
tokenHash	String	UNIQUE	SHA-256 hex of the raw session token
userId	String	FK → User	Cascade delete
expiresAt	DateTime	NOT NULL, INDEX	
createdAt	DateTime	DEFAULT now()	
lastUsedAt	DateTime	DEFAULT now()	Updated on each request

5.2.3 Assignment

Column	Type	Constraints	Description
id	String	PK	

title	String	NOT NULL	
language	AssignmentLanguage	NOT NULL	JAVA, C, or CPP
dueDate	DateTime?		Optional submission deadline
comparisonInputVersion	Int	DEFAULT 0	Incremented on input changes
professorId	String	FK → User	
createdAt / updatedAt	DateTime		

5.2.4 AssignmentKey

Column	Type	Constraints	Description
id	String	PK	
publicKey	String	UNIQUE	Random key shared with students
assignmentId	String	FK → Assignment	
isActive	Boolean	DEFAULT true	
createdAt	DateTime		

5.2.5 UploadBatch

Column	Type	Constraints	Description
id	String	PK	
assignmentId	String	FK → Assignment	
uploaderId	String?	FK → User	Null for public/anonymous uploads
encryptedIdentity	String?		sidenc:v1:... blob (public submissions)
purpose	UploadPurpose	NOT NULL	STUDENT_SUBMISSION, HISTORICAL_SUBMISSION, TEMPLATE_UPLOAD
status	UploadBatchStatus	DEFAULT RECEIVED	RECEIVED, PROCESSING, READY, FAILED
originalObjectKey	String		Storage key of the uploaded archive
errorMessage	String?		Set on FAILED
warningMessage	String?		Non-fatal issues (e.g., skipped files)
skippedSubmissionCount	Int	DEFAULT 0	Files skipped during extraction
createdAt / updatedAt	DateTime		

5.2.6 Submission

Column	Type	Constraints	Description
id	String	PK	
assignmentId	String	FK → Assignment	
uploadBatchId	String	FK → UploadBatch	
ownerId	String?	FK → User	Null for anonymous uploads
displayName	String	NOT NULL	Human-readable identifier
kind	SubmissionKind	NOT NULL	CURRENT or HISTORICAL
concatenatedSource	String	NOT NULL	Full concatenated source code
sourceMapJson	Json	NOT NULL	Array of SourceMapEntry
createdAt / updatedAt	DateTime		

5.2.7 SubmissionFile

Column	Type	Constraints	Description
id	String	PK	
submissionId	String	FK → Submission	
relativePath	String		Path within original archive
storageObjectKey	String		Key in object store
canonicalOrder	Int		Order for concatenation
byteStart	Int		Start offset in concatenated source
byteEnd	Int		End offset in concatenated source
archivePath	String?		Path within child zip if nested

5.2.8 AssignmentTemplate / TemplateFile

Analogous to Submission/SubmissionFile but for professor-uploaded template code. Includes a `versionNumber` and `isActive` flag to track the current template version.

5.2.9 ComparisonRun

Column	Type	Constraints	Description
id	String	PK	
assignmentId	String	FK → Assignment	
templateVersionId	String?	FK → AssignmentTemplate	Active template used
inputVersion	Int	DEFAULT 0	Snapshot of comparisonInputVersion
status	ComparisonRunStatus	DEFAULT QUEUED	QUEUED, RUNNING, COMPLETED, FAILED

engineVersion	String		Recorded for reproducibility
paramsJson	Json		gstMinMatchLength, minimumCommentLength
startedAt / completedAt	DateTime?		
errorMessage	String?		

5.2.10 PairResult

Column	Type	Constraints	Description
id	String	PK	
comparisonRunId	String	FK → ComparisonRun	
leftSubmissionId	String	FK → Submission	
rightSubmissionId	String	FK → Submission	
similarityScore	Float	NOT NULL	0.0–1.0
commentScore	Float?		Reserved
matchedTokenCount	Int	NOT NULL	
sortOrder	Int	NOT NULL	Pre-computed rank

5.2.11 PairMatch

Column	Type	Constraints	Description
id	String	PK	
pairResultId	String	FK → PairResult	
matchKey	String	UNIQUE per pairResult	e.g., code-0, comment-3
kind	MatchKind	NOT NULL	CODE or COMMENT
leftByteStart / leftByteEnd	Int		Byte offsets in left concatenated source
rightByteStart / rightByteEnd	Int		Byte offsets in right concatenated source
leftLineStart / leftLineEnd	Int?		Line numbers (1-based)
rightLineStart / rightLineEnd	Int?		
leftTokenStart / leftTokenEnd	Int?		Token indices
rightTokenStart / rightTokenEnd	Int?		

5.3 Entity Relationships

Listing 5.1: Entity relationship summary

```
User          1---* Assignment    (professor owns assignments)
User          1---* UploadBatch   (professor uploads batches)
User          1---* Submission    (student owns submissions, optional)
User          1---* Session       (cascade delete)

Assignment    1---* AssignmentKey
Assignment    1---* UploadBatch
Assignment    1---* Submission
Assignment    1---* AssignmentTemplate
Assignment    1---* ComparisonRun

UploadBatch   1---* Submission
UploadBatch   1---* AssignmentTemplate

Submission    1---* SubmissionFile
Submission    1---* PairResult (as left)
Submission    1---* PairResult (as right)

AssignmentTemplate 1---* TemplateFile
AssignmentTemplate 1---* ComparisonRun

ComparisonRun 1---* PairResult
PairResult    1---* PairMatch
```

5.4 Indexing Strategy

Defined indexes observed in the schema:

- `Session.tokenHash` — UNIQUE index for fast session lookup on every request.
- `Session.userId` — Index for cascading deletes.
- `Session.expiresAt` — Index for efficient expired session purging.
- `PairMatch(pairResultId,matchKey)` — UNIQUE composite index.
- `AssignmentKey.publicKey` — UNIQUE index for student key lookup.
- `User.email` — UNIQUE index for login lookup.

NOTE: Additional indexes on foreign keys (e.g., `UploadBatch.assignmentId`, `Submission.assignmentId`) would benefit query performance as data scales. These are not currently declared explicitly in the Prisma schema and rely on default behaviour.

5.5 Migration Approach

Migrations are managed by Prisma Migrate. Development: `npmrundb:migrate:dev` (generates and applies migration). Production: `npmrundb:migrate:deploy` (applies committed migrations only, no schema drift). Migration files are committed to version control in `prisma/migrations/`.

The migration history shows:

- `202604010001_init` — Initial schema
- `202604030001_public_student_uploads` — Public upload support
- `202604050001_assignment_due_date` — Due date field
- `202604070001_manual_comparison_runs` — Manual comparison run trigger
- `202604070001_professor_profile_fields` — Professor profile fields (title, department)
- `202604210001_upload_batch_warnings` — Warning message and skip count on batches

Chapter 6. Source Code Documentation

6.1 Module Inventory

Module path	Purpose	Key dependencies
apps/api/src/main.ts	Bootstrap NestJS/Fastify application, register plugins, bind port	NestJS, Fastify, cookie, multipart
apps/api/src/config/runtime-config.ts	Load and validate all API environment variables at startup	node:crypto (for dev key generation)
apps/api/src/modules/auth/*	Session-based authentication, role guards, password service	PrismaService, node:crypto
apps/api/src/modules/assignments/*	Assignment CRUD, maintenance utilities	PrismaService, storage
apps/api/src/modules/submissions/*	Upload batch management, identity encryption, downloads	PrismaService, QueueService, storage
apps/api/src/modules/comparisons/*	Comparison run creation and result retrieval	PrismaService, QueueService
apps/api/src/modules/health/*	Liveness and readiness probes	PrismaService, ioredis
apps/api/src/modules/queue/*	BullMQ queue publisher (upload-preparation, comparison-run)	BullMQ, ioredis
apps/api/src/modules/prisma/*	Singleton PrismaClient wrapper	Prisma Client
apps/worker/src/main.ts	Bootstrap worker: verify engine binary, mkdir storage, register BullMQ workers	
apps/worker/src/config/runtime-config.ts	Load and validate all worker environment variables	
apps/worker/src/runtime/redis.ts	Create ioredis connection for BullMQ	ioredis
apps/worker/src/runtime/workers.ts	Register both BullMQ Worker instances	BullMQ
apps/worker/src/services/upload-preparation.processor.ts	Job handler: orchestrates archive extraction, concatenation, persistence	
apps/worker/src/services/comparison-run.processor.ts	Job handler: pair generation, engine invocation, result persistence	
apps/worker/src/services/archive-extraction.service.ts	ZIP extraction with depth/size guards	fflate
apps/worker/src/services/concat.service.ts	Deterministic source concatenation and source map generation	
apps/worker/src/services/engine-runner.service.ts	Spawn engine binary, read/write JSON via stdio	node:child_process
apps/worker/src/services/pair-generation.service.ts	Generate current×current and current×historical pairs	
apps/worker/src/services/persist-comparison-results.service.ts	Upsert PairResult and PairMatch rows	Prisma
apps/worker/src/services/persist-prepared-upload.service.ts	Write Submission, SubmissionFile, AssignmentTemplate rows	Prisma
packages/shared/src/storage.ts	Filesystem object-storage abstraction	node:fs
packages/shared/src/queues.ts	Queue name constants	
packages/shared/src/engine.ts	TypeScript types for engine protocol	
packages/shared/src/domain.ts	Shared domain types	
engine/crates/engine-cli	CLI binary: read stdin, call analyze(), write stdout	serde_json
engine/crates/engine-contracts	Serialisable request/response types	serde
engine/crates/engine-core	Analysis orchestration, GST orchestration, scoring, comment matching	engine-gst, engine-language
engine/crates/engine-gst	Greedy String Tiling implementation	
engine/crates/engine-language	tree-sitter parsing, token normalisation adapters	tree-sitter-java, -c, -cpp

6.2 Key Algorithms Explained

6.2.1 Greedy String Tiling (GST)

The GST algorithm (Wise 1996) finds the longest common, non-overlapping substrings between two token sequences. In each round: scan all (left, right) starting positions for unmarked tokens, identify the maximum match length, collect all matches of that length, mark their tokens, and add them to the accepted tile set. Repeat until no match of at least `gstMinMatchLength` tokens remains.

Complexity is $O(n \cdot m)$ per round in the worst case, where n and m are the lengths of the two sequences. The number of rounds is bounded by the number of distinct tiles. For typical student submissions this is fast in practice.

6.2.2 Source Map and Byte Span Tracking

Source files are concatenated by the worker before storage. A `sourceMapJson` array records which byte range of the concatenated string corresponds to which original file path. The engine receives this and uses it to report match spans back to the original file paths. The pair viewer uses these spans to highlight matching regions in the correct source file panel.

6.2.3 Token Normalisation

Before GST, the engine tokenises source using tree-sitter and normalises tokens:

- Identifiers → `ID` (variable/function names)
- Type identifiers → `TYPE_ID`
- Field identifiers → `FIELD_ID`
- Numeric literals → `NUM` / `INT_LIT` / `FLOAT_LIT`
- String literals → `STR` / `STR_LIT`
- Java package/import, C preprocessor directives → skipped (unit gaps inserted)
- Comments → extracted separately into comment token list

6.3 Configuration Reference

Variable	Type	Default	Effect
NODE_ENV	string	development	Affects production-mode strictness checks
DATABASE_URL	string	(required)	PostgreSQL connection string
REDIS_URL	string	redis://localhost:6379	Redis connection string
API_HOST	string	0.0.0.0	API bind host
API_PORT	integer	3001	API bind port
CORS_ALLOWED_ORIGINS	CSV string	(dev: localhost)	Comma-separated allowed origins
UPLOAD_MAX_FILE_SIZE_MB	integer	250	Max upload size in megabytes
AUTH_SESSION_COOKIE_NAME	string	similarity_session	Cookie name
AUTH_SESSION_TTL_HOURS	integer	168 (7 days)	Session lifetime
AUTH_COOKIE_SECURE	bool	false (dev)	Secure cookie flag
AUTH_COOKIE_SAME_SITE	lax/strict/none	lax	SameSite cookie attribute
AUTH_COOKIE_DOMAIN	string	(unset)	Cookie domain scope
PUBLIC_UPLOAD_STATUS_TOKEN_SECRET	string	(required prod)	HMAC secret for public upload status tokens
SUBMISSION_IDENTITY_KEY_ID	string	(required prod)	Key ID for RSA identity key
SUBMISSION_IDENTITY_PUBLIC_KEY_PEM_BASE64	base64 PEM	(required prod)	RSA public key for student identity encryption
SUBMISSION_IDENTITY_PRIVATE_KEY_PEM_BASE64	base64 PEM	(required prod)	RSA private key for identity decryption
NEXT_PUBLIC_API_BASE_URL	string	(required)	API base URL for frontend
ENGINE_BINARY_PATH	string	(required prod)	Absolute path to engine-cli binary
ENGINE_VERSION	string	0.1.0	Recorded in ComparisonRun rows
ENGINE_GST_MIN_MATCH_LENGTH	integer	8	Minimum GST tile length
ENGINE_MINIMUM_COMMENT_LENGTH	integer	12	Minimum comment length for matching
WORKER_UPLOAD_CONCURRENCY	integer	2	Parallel upload preparation jobs
WORKER_COMPARISON_CONCURRENCY	integer	1	Parallel comparison run jobs
OBJECT_STORAGE_MODE	string	filesystem	Storage backend (only “filesystem” supported)
OBJECT_STORAGE_ROOT	path	./var/object-storage	Root directory for object storage
ARCHIVE_MAX_DEPTH	integer	8	Maximum ZIP nesting depth
ARCHIVE_MAX_SOURCE_FILES	integer	2000	Maximum files per archive
ARCHIVE_MAX_EXPANDED_BYTES	integer	52428800 (50 MB)	Maximum uncompressed bytes
ADMIN_BOOTSTRAP_EMAIL	string	(bootstrap only)	Email for first admin account
ADMIN_BOOTSTRAP_PASSWORD	string	(bootstrap only)	Password for first admin account

Chapter 7. Security Architecture

7.1 Authentication and Session Management

Session tokens are 32-byte cryptographically random values (`node:cryptorandomBytes`), encoded as base64url. Only the SHA-256 hash of the token is stored in the `Session` table. This ensures database compromise does not expose valid session tokens.

Sessions are set as httpOnly, Secure (in production), SameSite=Lax cookies. TTL defaults to 7 days. On logout, the session row is deleted. Expired sessions are deleted lazily on use.

7.2 Role-Based Access Control

Two roles: `professor` and `student`. The `SessionAuthGuard` (global) validates the session and populates `request.authUser`. The `RolesGuard` checks the `@Roles()` decorator metadata. Public routes bypass both guards via `@Public()`.

7.3 Submission Identity Privacy

Anonymous student submissions use RSA-OAEP-256 (RSA 4096-bit key, SHA-256 hash). The client fetches the public key (`GET/uploads/public/identity-key`), encrypts the identity JSON client-side, and sends the base64url ciphertext in the format `sidenc:v1:\{keyId\}:\{ciphertext\}`. The server stores the ciphertext blob, never plaintext. The professor explicitly requests decryption via `GET/uploads/assignment/:id/submissions/:id/identity`, which triggers server-side private-key decryption.

7.4 Path Traversal Protection

The storage module validates object keys: rejects empty strings, keys starting with `..`, or keys containing path components that are `...` File names are sanitised by replacing non-alphanumeric characters. Object keys are always resolved relative to the configured `OBJECT_STORAGE_ROOT`.

7.5 Input Validation

All HTTP request bodies are validated via NestJS DTO classes. Multipart uploads are limited by `UPLOAD_MAX_FILE_SIZE_MB`. Archive extraction enforces maximum depth (`ARCHIVE_MAX_DEPTH`), file count (`ARCHIVE_MAX_SOURCE_FILES`), and total expanded size (`ARCHIVE_MAX_EXPANDED_BYTES`).

7.6 CORS

CORS is enabled on the API with `credentials:true` and the allowed origins list from `CORS_ALLOWED_ORIGINS`. In production this must be explicitly configured; the default (localhost) is rejected.

NOTE: No rate limiting is implemented. For production deployments, rate limiting at the Caddy or application layer is strongly recommended, particularly on the public upload endpoints and the login endpoint.

Chapter 8. Testing Architecture

8.1 Test Types

- **Unit tests (API and Worker)**: Located as `*.test.ts` files alongside source. Test individual service methods in isolation. Files include: `assignments.service.test.ts`, `auth.controller.test.ts`, `auth.service.test.ts`, `comparisons.service.test.ts`, `comparison-status.test.ts`, `submissions.controller.test.ts`, `submissions.service.test.ts`, `archive-extraction.service.test.ts`, `child-submission-archive-layout.service.test.ts`, `concat.service.test.ts`, `prepare-upload-artifacts.service.test.ts`.
- **Rust unit tests (Engine)**: Inline `\#[cfg(test)]` modules in `engine-core/src/lib.rs` and `engine-language/src/lib.rs`. Comprehensive: cover GST scoring, similarity metrics, template masking, comment matching, normalisation, boilerplate exclusion, and specific Java/C/C++ language behaviours.
- **Shared package tests**: `packages/shared/src/storage.test.ts` tests the object storage helpers.

NOTE: No end-to-end (E2E) browser tests were found in the repository. No CI/CD pipeline configuration was found (no GitHub Actions workflows, no `.gitlab-ci.yml`, no Jenkinsfile). These represent significant gaps for a production deployment.

8.2 Running Tests

Listing 8.1: Run all tests

```
# TypeScript tests (all workspaces)
npm run test

# Rust engine tests
cargo test --manifest-path engine/Cargo.toml --workspace
```

Chapter 9. Build & Deployment

9.1 Local Development Setup

Listing 9.1: Local development setup steps

```
# 1. Clone repository
git clone <repo-url> && cd Anti-Vibe-Coder

# 2. Copy environment file
cp .env.example .env
# Edit .env: set DATABASE_URL, REDIS_URL, ENGINE_BINARY_PATH, etc.

# 3. Install Node dependencies
npm install

# 4. Build the Rust engine (requires Rust 1.86+)
cargo build --manifest-path engine/Cargo.toml

# 5. Generate Prisma client
npm run prisma:generate

# 6. Apply database migrations
npm run db:migrate:dev

# 7. Bootstrap first admin account
npm run db:bootstrap-admin

# 8. (Optional) Seed demo data
npm run db:seed

# 9. Start services (separate terminals):
npm run dev --workspace @similarity/api
npm run dev --workspace @similarity/worker
npm run dev --workspace @similarity/web
```

9.2 Build Process

Listing 9.2: Production build

```
# Build engine (Rust, release mode)
cargo build --manifest-path engine/Cargo.toml --release
# Output: engine/target/release/engine-cli

# Build all TypeScript packages
npm run prisma:generate
npm run build
```

```
# This runs: shared -> api -> worker -> web (in order)
```

9.3 Docker Containerised Deployment

Listing 9.3: Container deployment

```
# Build and start all services
docker compose up --build
```

The `docker-compose.yml` starts: `postgres`, `redis`, `api`, `worker`, `web`, `caddy`.

The object storage volume (`object-storage`) is shared between `api` and `worker` containers. This is a Docker named volume; data persists across container restarts.

The `Dockerfile.worker` uses a multi-stage build: Stage 1 builds the Rust engine binary with Rust 1.86; Stage 2 builds the Node.js worker; Stage 3 copies both into the final image. The engine binary is placed at `/app/bin/engine-cli`.

9.4 Deployment Bootstrap Order

1. Set all required environment variables (see configuration reference).
2. Run `npmrunprisma:generate` to generate the Prisma Client.
3. Run `npmrundb:migrate:deploy` to apply all committed migrations.
4. Run `npmrundb:bootstrap-admin` to create the first professor account.
5. Optionally run `npmrundb:seed` in non-production environments.
6. Build and deploy web, api, worker, and engine containers.
7. Validate `GET/health/live` returns 200 and `GET/health/ready` returns 200 with `database:"ok"` and `redis:"ok"`.
8. Perform smoke-test: login, assignment creation, upload, comparison run, pair viewer.

9.5 Environment Differences

Concern	Development	Production
RSA key pair	Auto-generated at startup	Must be set via env vars
Cookie Secure flag	false	true
CORS origins	localhost defaults	Must be set explicitly
Engine binary path	Resolves to `engine/target/debug/engine-cli.exe`	Must be set to absolute path
DATABASE_URL	Optional fallback	Required
Redis URL	redis://localhost:6379	Required
Node ENV	development	production